数据库系统概论

An Introduction to Database System

第三章 关系数据库标准语言SQL(续2)

山西大学计算机与信息技术学院

第三章 关系数据库标准语言SQL

3.1 SQL概述

3.2 学生-课程数据库

3.3 数据定义

3.4 数据查询

3.5 数据更新

3.6 视图

3.7 小结

3.5 数据更新

3.5.1　插入数据

3.5.2　修改数据

3.5.3　删除数据

3.5.1 插入数据

- 两种插入数据方式

1. 插入元组
2. 插入子查询结果

- 可以一次插入多个元组

一、插入元组

- 语句格式

  INSERT

  INTO <表名> [(<属性列1>[, <属性列2 >...)]

  VALUES (<常量1> [, <常量2>] ...)

- 功能

  - 将新元组插入指定表中

插入元组（续）

- INTO子句
  - 属性列的顺序可与表定义中的顺序不一致
  - 没有指定属性列
  - 指定部分属性列
- VALUES子句
  - 提供的值必须与INTO子句匹配
    - 值的个数
    - 值的类型

插入元组（续）

［例1］　将一个新学生元组（学号：200215128；姓名：陈冬；性别：男；所在系：IS；年龄：18岁）插入到Student表中。

```
INSERT
INTO  Student (Sno，Sname，Ssex，Sdept，Sage)
VALUES ('200215128'，'陈冬'，'男'，'IS'，18);
```

插入元组（续）

［例2］　将学生张成民的信息插入到Student表中。

```
INSERT
INTO  Student
VALUES（‘200215126’，　‘张成民’，　‘男’，18，’CS’）;
```

插入元组（续）

[例3]　插入一条选课记录(’200215128’,’1 ’)。

```
INSERT
INTO SC(Sno, Cno)
VALUES (‘ 200215128 ’ , ‘ 1 ’ );
```

RDBMS将在新插入记录的Grade列上自动地赋空值。

或者：

```
INSERT
INTO SC
VALUES (’ 200215128 ’,’ 1 ’, NULL);
```

二、插入子查询结果

- 语句格式

  INSERT

  INTO <表名> [(<属性列1>[,<属性列2>...)]

  子查询;

- 功能

  将子查询结果插入指定表中

插入子查询结果（续）

- INTO子句(与插入元组类似)
- 子查询
  - SELECT子句目标列必须与INTO子句匹配
    - 值的个数
    - 值的类型

插入子查询结果（续）

[例4]对每一个系，求学生的平均年龄，并把结果存入数据库。

第一步：建表

```
CREATE  TABLE  Dept_age
    (Sdept  CHAR(15)                    /* 系名*/
     Avg_age SMALLINT);            /*学生平均年龄*/
```

插入子查询结果（续）

第二步：插入数据

```
INSERT
 INTO  Dept_age(Sdept，Avg_age)
      SELECT  Sdept，AVG(Sage)
      FROM  Student
      GROUP BY Sdept;
```

插入子查询结果（续）

RDBMS在执行插入语句时会检查所插元组是否破坏表上已定义的完整性规则

- 实体完整性
- 参照完整性
- 用户定义的完整性
  - NOT NULL约束
  - UNIQUE约束
  - 值域约束

3.5 数据更新

3.5.1　插入数据

3.5.2　修改数据

3.5.3　删除数据

3.4.2 修改数据

- 语句格式

  UPDATE <表名>
  SET <列名>=<表达式>[，<列名>=<表达式>]...
  [WHERE <条件>];

- 功能

  - 修改指定表中满足WHERE子句条件的元组

修改数据（续）

- SET子句
  - 指定修改方式
  - 要修改的列
  - 修改后取值
- WHERE子句
  - 指定要修改的元组
  - 缺省表示要修改表中的所有元组

修改数据（续）

- 三种修改方式

1. 修改某一个元组的值
2. 修改多个元组的值
3. 带子查询的修改语句

1. 修改某一个元组的值

[例5]　将学生200215121的年龄改为22岁

```
UPDATE  Student
SET Sage=22
WHERE  Sno=’200215121’;
```

2. 修改多个元组的值

[例6]　将所有学生的年龄增加1岁

```
UPDATE Student
SET Sage= Sage+1;
```

3. 带子查询的修改语句

[例7]　将计算机科学系全体学生的成绩置零。

```
UPDATE SC
SET  Grade=0
WHERE  Sno IN
        (SELETE Sno
         FROM  Student
         WHERE  Sdept = 'CS');
```

修改数据（续）

RDBMS在执行修改语句时会检查修改操作是否破坏表上已定义的完整性规则

- 实体完整性
- 主码不允许修改
- 用户定义的完整性
  - NOT NULL约束
  - UNIQUE约束
  - 值域约束

3.5 数据更新

3.5.1 插入数据

3.5.2 修改数据

3.5.3 删除数据

3.5.3 删除数据

- 语句格式

  DELETE

  FROM　　〈表名〉

  [WHERE 〈条件〉]；

- 功能
  - 删除指定表中满足WHERE子句条件的元组
- WHERE子句
  - 指定要删除的元组
  - 缺省表示要删除表中的全部元组，表的定义仍在字典中

删除数据（续）

- 三种删除方式

1. 删除某一个元组的值
2. 删除多个元组的值
3. 带子查询的删除语句

1. 删除某一个元组的值

[例8]　删除学号为200215128的学生记录。

```
DELETE
 FROM Student
 WHERE Sno= 200215128 ';
```

2. 删除多个元组的值

[例9]　删除所有的学生选课记录。

```
DELETE
FROM SC;
```

3. 带子查询的删除语句

[例10]　删除计算机科学系所有学生的选课记录。

```
DELETE
FROM SC
WHERE  Sno IN
          (SELETE Sno
           FROM  Student
           WHERE  Sdept = 'CS');
```

第三章 关系数据库标准语言SQL

3.1 SQL概述

3.2 学生-课程数据库

3.3 数据定义

3.4 数据查询

3.5 数据更新

3.6 视图

3.7 小结

3.6 视 图

视图的特点

- 虚表，是从一个或几个基本表（或视图）导出的表
- 只存放视图的定义，不存放视图对应的数据
- 基表中的数据发生变化，从视图中查询出的数据也随之改变

3.6 视　图

基于视图的操作

- 查询
- 删除
- 受限更新
- 定义基于该视图的新视图

3.6 视　图

3.6.1　定义视图

3.6.2　查询视图

3.6.3　更新视图

3.6.4　视图的作用

3.6.1 定义视图

- 建立视图
- 删除视图

一、建立视图

- 语句格式

  CREATE VIEW

  <视图名> [(<列名> [，<列名>]...)]

  AS <子查询>

  [WITH CHECK OPTION]；

- 组成视图的属性列名：全部省略或全部指定

建立视图（续）

- RDBMS执行CREATE VIEW语句时只是把视图定义存入数据字典，并不执行其中的SELECT语句。
- 在对视图查询时，按视图的定义从基本表中将数据查出。

建立视图（续）

[例1]　建立信息系学生的视图。

```
CREATE VIEW IS_Student
AS
SELECT Sno，Sname，Sage
FROM    Student
WHERE  Sdept= 'IS'；
```

建立视图（续）

[例2]建立信息系学生的视图，并要求进行修改和插入操作时仍需保证该视图只有信息系的学生 。

```
CREATE VIEW IS_Student
AS
SELECT Sno，Sname，Sage
FROM  Student
WHERE  Sdept= 'IS'
WITH CHECK OPTION;
```

建立视图（续）

对IS_Student视图的更新操作：

- 修改操作：自动加上Sdept= ’IS’的条件
- 删除操作：自动加上Sdept= ’IS’的条件
- 插入操作：自动检查Sdept属性值是否为’IS’

  如果不是，则拒绝该插入操作

  如果没有提供Sdept属性值，则自动定义Sdept为’IS’

建立视图（续）

- 基于多个基表的视图

[例3]　建立信息系选修了1号课程的学生视图。

```
CREATE VIEW IS_S1(Sno，Sname，Grade)
AS
SELECT Student.Sno，Sname，Grade
FROM  Student，SC
WHERE  Sdept='IS' AND
                Student.Sno=SC.Sno AND
                SC.Cno='1';
```

建立视图（续）

- 基于视图的视图

[例4]　建立信息系选修了1号课程且成绩在90分以上的学生的视图。

```
CREATE VIEW IS_S2
AS
SELECT Sno，Sname，Grade
FROM  IS_S1
WHERE  Grade>=90;
```

建立视图（续）

- 带表达式的视图

[例5]　定义一个反映学生出生年份的视图。

```
CREATE  VIEW BT_S(Sno，Sname，Sbirth)
AS
SELECT Sno，Sname，2015-Sage
FROM  Student;
```

建立视图（续）

- 分组视图

[例6]　将学生的学号及他的平均成绩定义为一个视图

假设SC表中“成绩”列Grade为数字型

```
CREAT  VIEW S_G(Sno，Gavg)
AS
SELECT Sno，AVG(Grade)
FROM  SC
GROUP BY Sno;
```

建立视图（续）

- 不指定属性列

[例7]将Student表中所有女生记录定义为一个视图

```
CREATE VIEW F_Student(F_Sno，name，sex，age，dept)
AS
SELECT *
FROM  Student
WHERE Ssex=‘女’；
```

缺点：

修改基表Student的结构后，Student表与F_Student视图的映象关系被破坏，导致该视图不能正确工作。

二、删除视图

- 语句的格式：

  DROP VIEW 〈视图名〉;

  该语句从数据字典中删除指定的视图定义

  如果该视图上还导出了其他视图，使用CASCADE级联删除语句，把该视图和由它导出的所有视图一起删除

  删除基表时，由该基表导出的所有视图定义都必须显式地使用DROP VIEW语句删除

删除视图(续）

［例8］　删除视图BT_S：　DROP VIEW BT_S;

删除视图IS_S1：DROP VIEW IS_S1;

- 拒绝执行
- 级联删除：

DROP VIEW IS_S1 CASCADE;

3.6 视 图

3.6.1 定义视图

3.6.2 查询视图

3.6.3 更新视图

3.6.4 视图的作用

3.6.2 查询视图

- 用户角度：查询视图与查询基本表相同
- RDBMS实现视图查询的方法

  视图消解法（View Resolution）

  - 进行有效性检查
  - 转换成等价的对基本表的查询
  - 执行修正后的查询

查询视图（续）

[例9]　在信息系学生的视图中找出年龄小于20岁的学生。

```
SELECT    Sno，Sage
FROM      IS_Student
WHERE     Sage<20;
```

查询视图（续）

视图消解转换后的查询语句为：

```
SELECT  Sno，Sage
FROM  Student
WHERE  Sdept= 'IS'  AND  Sage<20;
```

查询视图（续）

[例10]　查询选修了1号课程的信息系学生

```
SELECT   IS_Student.Sno, Sname
FROM      IS_Student, SC
WHERE   IS_Student.Sno =SC.Sno AND SC.Cno= '1';
```

查询视图（续）

- 视图消解法的局限

  有些情况下，直接进行视图消解法不能生成正确查询。

查询视图（续）

[例11]在S_G视图中查询平均成绩在90分以上的学生学号和平均成绩

```
SELECT *
FROM  S_G
WHERE  Gavg>=90;
```

S_G视图的子查询定义：

```
CREATE VIEW S_G (Sno，Gavg)
AS
SELECT  Sno，AVG(Grade)
 FROM  SC
GROUP BY Sno;
```

查询转换

错误：

```
SELECT Sno，AVG(Grade)
FROM     SC
WHERE  AVG(Grade)>=90
GROUP BY Sno;
```

正确：

```
SELECT  Sno，AVG(Grade)
FROM  SC
GROUP BY Sno
HAVING AVG(Grade)>=90;
```

3.6 视　图

3.6.1　定义视图

3.6.2　查询视图

3.6.3　更新视图

3.6.4　视图的作用

更新视图（续）

［例12］ 将信息系学生视图IS_Student中学号200215122的学生姓名改为“刘辰”。

```
UPDATE  IS_Student
SET  Sname= '刘辰'
WHERE  Sno= ' 200215122 ';
```

转换后的语句：

```
UPDATE  Student
SET Sname= '刘辰'
WHERE Sno= ' 200215122 ' AND Sdept= 'IS';
```

更新视图（续）

[例13]　向信息系学生视图IS_S中插入一个新的学生记录：200215129，赵新，20岁

```
INSERT
INTO IS_Student
VALUES(‘95029’，‘赵新’，20);
```

转换为对基本表的更新：

```
INSERT
INTO   Student(Sno，Sname，Sage，Sdept)
VALUES(‘200215129’，’赵新’，20，’IS’);
```

更新视图（续）

[例14]删除信息系学生视图IS_Student中学号为200215129的记录

```
DELETE
FROM IS_Student
WHERE Sno= ' 200215129 ';
```

转换为对基本表的更新：

```
DELETE
FROM Student
WHERE Sno= ' 200215129 ' AND Sdept= 'IS';
```

更新视图（续）

- 更新视图的限制：一些视图是不可更新的，因为对这些视图的更新不能唯一地有意义地转换成对相应基本表的更新

例：视图S_G为不可更新视图。

```
UPDATE  S_G
SET     Gavg=90
WHERE   Sno= '200215121';
```

这个对视图的更新无法转换成对基本表SC的更新

更新视图（续）

- 允许对行列子集视图进行更新
- 对其他类型视图的更新不同系统有不同限制

3.6 视　图

3.6.1　定义视图

3.6.2　查询视图

3.6.3　更新视图

3.6.4　视图的作用

3.6.4 视图的作用

- 1. 视图能够简化用户的操作
- 2. 视图使用户能以多种角度看待同一数据
- 3. 视图对重构数据库提供了一定程度的逻辑独立性
- 4. 视图能够对机密数据提供安全保护
- 5. 适当的利用视图可以更清晰的表达查询

3.7 小结

- 1. SQL语言的特点
- 2. 数据定义语言
- 3. 数据查询语言
- 4. 数据更新语言
- 5. 视图

下课了。。。

休息一会儿。。。